

Feuille de travaux dirigés n° 5

Algorithmes d'exclusion mutuelle

Exercice 1 (L'algorithme de filtre ¹).

On considère l'algorithme suivant, pour 3 threads d'identifiants 0, 1 et 2.

```
class PetersonFor3 {  
    private final AtomicIntegerArray entering = new AtomicIntegerArray(3);  
    private volatile int lastArrived;  
    public void lock() {  
        int i = ThreadID.get();  
        entering.set(i, 1);  
        lastArrived = i;  
        for(int j = 0; j < 3; j++) {  
            while(j != i && entering.get(j) == 1 && lastArrived == i);  
        }  
    }  
    public void unlock() {  
        int i = ThreadID.get();  
        entering.set(i, 0);  
    }  
}
```

- 1a. Montrez qu'au plus deux threads peuvent accéder en même temps en section critique.
- 1b. Combinez cet algorithme à l'algorithme de Peterson pour concevoir un verrou entre trois threads.
- 1c. Généralisez à n threads.

Exercice 2 (Verrou réentrant).

On rappelle qu'un verrou est réentrant si un thread qui a déjà pris un verrou peut ré-entrer en section critique, ce qui signifie qu'il ne sera pas bloqué s'il appelle `lock()` à nouveau. Pour relâcher le verrou, un thread doit appeler `unlock()` autant de fois qu'il a appelé `lock()`.

- 2a. On se donne une implémentation non réentrante des verrous. La transformer en une implémentation réentrante.

Exercice 3 (L'instruction `getAndIncrement` ²).

On rappelle que la méthode `AtomicInteger.getAndIncrement()` incrémente le contenu de son instance, et retourne la valeur écrasée de manière atomique.

- 3a. Proposez une implémentation d'un verrou *starvation-free* en utilisant l'instruction spéciale `getAndIncrement` et capable de supporter un nombre de threads inconnu lors de sa construction.

1. Algorithme : J. L. M. Kessels. *On the existence of wait-free protocols for mutual exclusion in hard real-time environments*. IPL (1982)
2. Algorithme : J. M. Mellor-Crummey, M. L. Scott. *Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors*. TOCS (1991)

Exercice 4 (Verrou à *lecture/écriture*).

On rappelle qu'un verrou à *lecture/écriture* peut être pris soit en mode lecture (partagé entre tous les lecteurs), soit en mode écriture (exclusif à l'écrivain). Plusieurs lecteurs peuvent exécuter leur section critique de manière concurrente, mais un écrivain doit toujours être seul en section critique.

- 4a.** En vous inspirant de l'algorithme de la boulangerie, proposez une implémentation des verrous à lectures et écritures.